Practical no 7

Aim: Develop a program to manage resource allocation for five processes (Google Drive, Firefox, Word Processor, Excel, and PowerPoint) using four types of resources (Printer, ROM, Hard Disk, and RAM). The program takes input for allocated, maximum, and available resources, calculates the current need of each process, and determines if a safe execution order exists using the Banker's Algorithm.

Code:

```
7                                                   banker.c
io.h>

 // Number of processes
 // Number of resources


[P][R], max[P][R], need[P][R];
 [R];
h[P] = {0};
Seq[P];
 k;
 = 0;

Enter Allocation Matrix (5x4):\n");
; i < P; i++)
 = 0; j < R; j++)
scanf("%d", &alloc[i][j]);

Enter Maximum Matrix (5x4):\n");
; i < P; i++)
 = 0; j < R; j++)
scanf("%d", &max[i][j]);

Enter Available Resources (4 values):\n");
; i < R; i++)
f("%d", &avail[i]);

ate Need matrix
; i < P; i++)
 = 0; j < R; j++)
need[i][j] = max[i][j] - alloc[i][j];

nt < P) {
found = 0;
 = 0; i < P; i++) {
f(finish[i] == 0) {
    int flag = 0;
    for(j = 0; j < R; j++) {
        if(need[i][j] > avail[j]) {
            flag = 1;
            break;
        }

^G Write Out   ^F Where Is   ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To
^R Read File   ^\ Replace    ^U Paste     ^J Justify   ^/ Go To Line  M-E Redo      M-6 Copy       ^B Whe
```

```
GNU nano 8.7                                        banker.c
        for(i = 0; i < P; i++) {
            if(finish[i] == 0) {
                int flag = 0;
                for(j = 0; j < R; j++) {
                    if(need[i][j] > avail[j]) {
                        flag = 1;
                        break;
                    }
                }
                if(flag == 0) {
                    safeSeq[count++] = i;
                    for(k = 0; k < R; k++)
                        avail[k] += alloc[i][k];
                    finish[i] = 1;
                    found = 1;
                }
            }
        }
        if(found == 0) {
            printf("\nSystem is NOT in Safe State!\n");
            return 0;
        }
    }

    printf("\nSystem is in Safe State.\nSafe Sequence is:\n");

    char *processName[5] = {
        "Google Drive",
        "Firefox",
        "Word Processor",
        "Excel",
        "PowerPoint"
    };

    for(i = 0; i < P; i++)
        printf("%s -> ", processName[safeSeq[i]]);

    printf("END\n");

    return 0;
}
```

Output:

```
KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ cd

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ nano banker.c

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ gcc banker.c -o banker

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$ ./banker
Enter Allocation Matrix (5x4):
0 1 0 3
2 0 0 1
3 0 2 1
2 1 1 0
0 0 2 2

Enter Maximum Matrix (5x4):
7 5 3 4
3 2 2 2
9 0 2 2
2 2 2 2
4 3 3 3

Enter Available Resources (4 values):
3 3 2 2

System is in Safe State.
Safe Sequence is:
Firefox -> Excel -> PowerPoint -> Google Drive -> Word Processor -> END

KHUSHABU@DESKTOP-VE8LIPH MSYS ~
$
```